

Modern ES Features (ES2022–2024)

Modern JavaScript-এর কিছু গুরুত্বপূর্ণ built-in feature যা code-কে আরও readable, expressive, safe এবং practical করে।

Topics Covered

1. `Object.groupBy()` — ES2024
2. `Array.prototype.at()` — Negative Indexing
3. `Object.hasOwn()` — ES2022
4. `Error.cause` — Error Chaining
5. `structuredClone()` — Deep Cloning
6. Intl API Basics
7. `findLast()` & `findLastIndex()`
8. `replaceAll()`

1. `Object.groupBy()` — ES2024

What is `Object.groupBy()`?

`Object.groupBy()` কোনো iterable-এর elements-কে কোনো condition/key অনুযায়ী group করে একটি object তৈরি করে।

আপে সাধারণত `reduce()` দিয়ে এটা করতে হতো।

Old approach

```
const users = [
  { name: "Rahim", role: "admin" },
  { name: "Karim", role: "user" },
  { name: "Hasan", role: "admin" },
];

const grouped = users.reduce((result, user) => {
  if (!result[user.role]) {
    result[user.role] = [];
  }

  result[user.role].push(user);

  return result;
}, {});
```

```
}, {}));  
  
console.log(grouped);
```

Modern approach:

```
const grouped = Object.groupBy(users, user => user.role);  
  
console.log(grouped);
```

Result conceptually:

```
{  
  admin: [  
    { name: "Rahim", role: "admin" },  
    { name: "Hasan", role: "admin" }  
  ],  
  user: [  
    { name: "Karim", role: "user" }  
  ]  
}
```

Basic Syntax

```
Object.groupBy(iterable, callbackFn);
```

Example:

```
const numbers = [1, 2, 3, 4, 5, 6];  
  
const grouped = Object.groupBy(numbers, number => {  
  return number % 2 === 0 ? "even" : "odd";  
});  
  
console.log(grouped);
```

Result:

```
{  
  even: [2, 4, 6],
```

```
    odd: [1, 3, 5]
  }
```

Real-world Example

```
const products = [
  { name: "Laptop", category: "electronics" },
  { name: "Phone", category: "electronics" },
  { name: "Shirt", category: "clothing" },
  { name: "Pants", category: "clothing" },
];

const productsByCategory = Object.groupBy(
  products,
  product => product.category
);

console.log(productsByCategory);
```

Result:

```
{
  electronics: [
    { name: "Laptop", category: "electronics" },
    { name: "Phone", category: "electronics" }
  ],
  clothing: [
    { name: "Shirt", category: "clothing" },
    { name: "Pants", category: "clothing" }
  ]
}
```

Important: Callback কী return করে?

Callback যে value return করবে, সেটাই group key হবে।

```
const users = [
  { name: "A", age: 20 },
  { name: "B", age: 30 },
```

```
{ name: "C", age: 20 }  
];  
  
const result = Object.groupBy(users, user => user.age);
```

এখানে keys হবে:

```
"20"  
"30"
```

Object.groupBy() কী return করে?

এটি একটি **null-prototype object** return করে। অর্থাৎ normal `Object.prototype` inheritance থাকে না।

তাই result-কে normal object ধরে কিছু prototype method থাকার assumption করা উচিত নয়।

Object.groupBy() vs reduce()

Feature	Object.groupBy()	reduce()
Grouping	Built-in	Manually করতে হয়
Readability	বেশি	তুলনামূলক কম
Boilerplate	কম	বেশি
Custom transformation	সীমিত	অনেক flexible
Modern code	Recommended for grouping	General-purpose

Rule

শুধু grouping দরকার হলে:

```
Object.groupBy()
```

Complex accumulation/transformation দরকার হলে:

```
reduce()
```

2. `Array.prototype.at()` — Negative Indexing

Problem

Array-এর শেষ element পেতে traditionally:

```
const fruits = ["apple", "banana", "mango"];

console.log(fruits[fruits.length - 1]);
```

Modern JavaScript:

```
console.log(fruits.at(-1));
```

`at()` positive এবং negative দুটো index-ই support করে। Negative index array-এর শেষ দিক থেকে count করে।

Positive Index

```
const numbers = [10, 20, 30, 40];

console.log(numbers.at(0)); // 10
console.log(numbers.at(1)); // 20
console.log(numbers.at(3)); // 40
```

Negative Index

```
console.log(numbers.at(-1)); // 40
console.log(numbers.at(-2)); // 30
console.log(numbers.at(-3)); // 20
```

Mental model:

10	20	30	40
↑	↑	↑	↑
0	1	2	3

-4 -3 -2 -1

Last Element

```
const users = ["Rahim", "Karim", "Hasan"];  
  
const lastUser = users.at(-1);  
  
console.log(lastUser);
```

Output:

Hasan

Second Last

```
console.log(users.at(-2));
```

Output:

Karim

Out of Range

```
console.log(users.at(100));
```

Output:

undefined

`at()` vs Bracket Notation

```
arr[0];
```

vs

```
arr.at(0);
```

দুটাই valid ।

কিন্তু:

```
arr[-1];
```

এটা last element দেয় না ।

```
const arr = [10, 20, 30];  
  
console.log(arr[-1]);
```

Output:

```
undefined
```

কারণ `-1` এখানে normal property name হিসেবে treat হয় ।

তাই last item:

```
arr.at(-1);
```

`at()` শুধু **Array**-এর জন্য নয়

`at()` strings এবং অন্যান্য indexed collections-এর ক্ষেত্রেও ব্যবহার করা যায় ।

```
const text = "JavaScript";  
  
console.log(text.at(-1));
```

Output:

```
t
```

3. Object.hasOwn() — ES2022

Problem

আপে object-এর own property check করতে:

```
obj.hasOwnProperty("name");
```

কিন্তু এই approach-এর সমস্যা আছে।

Problem 1 — Property override

```
const user = {  
  name: "Shariar",  
  
  hasOwnProperty() {  
    return false;  
  }  
};  
  
console.log(user.hasOwnProperty("name"));
```

এখানে unexpected result পাওয়া যেতে পারে।

Problem 2 — Null prototype

```
const user = Object.create(null);  
  
user.name = "Shariar";
```

```
user.hasOwnProperty("name");
```

এখানে `hasOwnProperty` exist-ই করে না।

Modern Solution

```
Object.hasOwn(object, property);
```

Example:

```
const user = {  
  name: "Sharir",  
  age: 31  
};  
  
console.log(Object.hasOwn(user, "name"));
```

Output:

```
true
```

Missing Property

```
console.log(Object.hasOwn(user, "email"));
```

Output:

```
false
```

Inherited Property

```
const user = {  
  name: "Sharir"  
};
```

```
console.log(Object.hasOwn(user, "toString"));
```

Output:

```
false
```

কারণ `toString` inherited property, own property নয়।

`Object.hasOwn()` শুধু object's **own property** check করে।

Object.hasOwn() vs in

খুব important difference।

```
const user = {  
  name: "Sharlar"  
};  
  
console.log("name" in user);
```

```
true
```

কিন্তু:

```
console.log("toString" in user);
```

এটাও:

```
true
```

কারণ `in` prototype chain-ও check করে।

কিন্তু:

```
Object.hasOwn(user, "toString");
```

returns:

```
false
```

Summary

```
in
↓
own + inherited

Object.hasOwn()
↓
own only
```

Property value `undefined` হলেও?

```
const user = {
  name: undefined
};

console.log(Object.hasOwn(user, "name"));
```

Output:

```
true
```

কারণ property আছে।

এটা:

```
user.name === undefined
```

এর থেকে completely different concept।

4. Error `cause`

Modern JavaScript-এ একটি error-এর সাথে original error attach করা যায়।

```
new Error(message, { cause });
```

Problem

ধরো database operation fail করেছে।

```
try {  
    // database operation  
} catch (error) {  
    throw new Error("Failed to load user");  
}
```

এখানে original error হারিয়ে যেতে পারে।

Better:

```
try {  
    // database operation  
} catch (error) {  
    throw new Error("Failed to load user", {  
        cause: error  
    });  
}
```

এখন original error:

```
error.cause
```

এর মধ্যে থাকবে।

Example

```
try {  
    JSON.parse("invalid json");  
} catch (error) {  
    throw new Error("Failed to process configuration", {  
        cause: error  
    });  
}
```

```
});  
}
```

Reading Cause

```
try {  
  // ...  
} catch (error) {  
  console.log(error.message);  
  console.log(error.cause);  
}
```

Conceptually:

```
High-level error  
  ↓  
"Failed to process configuration"  
  ↓  
cause  
  ↓  
Original SyntaxError
```

Why `cause` is useful?

Layered application-এ খুব useful।

```
Database  
  ↓  
Repository  
  ↓  
Service  
  ↓  
Controller  
  ↓  
API response
```

Low-level error:

Connection timeout

Service layer:

```
throw new Error("Failed to create user", {  
  cause: error  
});
```

এতে high-level context + original debugging information দুটোই রাখা যায়।

Nested Causes

Cause chain তৈরি করা যায়:

```
try {  
  try {  
    throw new Error("Database connection failed");  
  } catch (error) {  
    throw new Error("Repository operation failed", {  
      cause: error  
    });  
  }  
} catch (error) {  
  throw new Error("User creation failed", {  
    cause: error  
  });  
}
```

Conceptually:

```
User creation failed  
  ↓  
Repository operation failed  
  ↓  
Database connection failed
```

5. structuredClone() — Deep Cloning

Problem

এটা:

```
const user2 = user1;
```

clone করে না।

এটা একই object-এর reference দেয়।

```
const user1 = {  
  name: "Sharif"  
};  
  
const user2 = user1;  
  
user2.name = "Rahim";  
  
console.log(user1.name);
```

Output:

Rahim

কারণ:

```
user1 ┌  
      └─> same object  
user2 ┌
```

Shallow Copy

```
const user2 = { ...user1 };
```

এটা top-level copy করে।

কিন্তু nested object-এর ক্ষেত্রে:

```
const user1 = {  
  name: "Shariar",  
  address: {  
    city: "Rangpur"  
  }  
};  
  
const user2 = { ...user1 };  
  
user2.address.city = "Dhaka";  
  
console.log(user1.address.city);
```

Output:

Dhaka

কারণ nested object-এর reference shared ।

Deep Clone with `structuredClone()`

```
const user1 = {  
  name: "Shariar",  
  address: {  
    city: "Rangpur"  
  }  
};  
  
const user2 = structuredClone(user1);  
  
user2.address.city = "Dhaka";  
  
console.log(user1.address.city);
```

Output:

Rangpur

এবার nested object-ও আলাদা clone হয়েছে।

`structuredClone()` structured clone algorithm ব্যবহার করে deep clone তৈরি করে এবং circular references-ও handle করতে পারে।

Circular Reference

JSON cloning-এর বড় সমস্যা:

```
const user = {  
  name: "Sharlar"  
};  
  
user.self = user;
```

এটা circular reference।

```
user  
↓  
self  
↓  
user  
↓  
self  
...
```

`JSON.stringify()` এখানে fail করবে।

কিন্তু:

```
const clone = structuredClone(user);
```

circular structure handle করতে পারে।

`structuredClone()` vs JSON Clone

Old hack:

```
const clone = JSON.parse(JSON.stringify(original));
```

এটার অনেক limitation আছে।

যেমন:

- undefined
- BigInt
- functions
- circular references
- কিছু special object type
- Date /other object semantics

Modern approach:

```
const clone = structuredClone(original);
```

structuredClone() কী clone করতে পারে?

Common supported types-এর মধ্যে আছে:

- Object
- Array
- Date
- Map
- Set
- RegExp
- ArrayBuffer
- TypedArray
- Error
- বিভিন্ন primitive values

তবে সব JavaScript value cloneable নয়। Function এবং DOM node clone করা যায় না এবং `DataCloneError` হতে পারে। Prototype chain, property descriptors, getters/setters-এর মতো metadata-ও সাধারণভাবে preserve হয় না।

Function Clone করা যায় না

```
const obj = {  
  name: "Sharlar",  
  greet() {  
    console.log("Hello");  
  }  
};
```

```
    }  
};  
  
structuredClone(obj);
```

Function থাকলে cloning fail করতে পারে।

Important: Deep Clone $\neq$ Everything Preserved

`structuredClone()` value/data structure clone করে।

এটা:

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

এর custom prototype/class behavior preserve করার general-purpose mechanism নয়।

তাই:

`structuredClone()` হলো data cloning-এর জন্য, arbitrary class instance-এর full semantic duplication-এর জন্য নয়।

6. Intl API Basics

`Intl` হলো JavaScript-এর internationalization API।

এটি language/locale অনুযায়ী:

- Number formatting
- Currency formatting
- Date formatting
- Time formatting
- Relative time
- Collation
- Plural rules

ইত্যাদি করতে সাহায্য করে।

Intl.NumberFormat

Basic

```
const formatter = new Intl.NumberFormat("en-US");  
  
console.log(formatter.format(1234567.89));
```

Output:

```
1,234,567.89
```

Bangladesh Locale

```
const formatter = new Intl.NumberFormat("bn-BD");  
  
console.log(formatter.format(1234567.89));
```

Locale অনুযায়ী formatting পরিবর্তিত হতে পারে।

Currency

```
const formatter = new Intl.NumberFormat("en-US", {  
  style: "currency",  
  currency: "USD"  
});  
  
console.log(formatter.format(1234.5));
```

Output conceptually:

```
$1,234.50
```

BDT

```
const formatter = new Intl.NumberFormat("en-BD", {  
  style: "currency",  
  currency: "BDT"  
});  
  
console.log(formatter.format(50000));
```

Percentage

```
const formatter = new Intl.NumberFormat("en-US", {  
  style: "percent"  
});  
  
console.log(formatter.format(0.75));
```

Output:

75%

Intl.DateTimeFormat

Date format locale অনুযায়ী generate করতে:

```
const date = new Date();  
  
const formatter = new Intl.DateTimeFormat("en-US");  
  
console.log(formatter.format(date));
```

Options

```
const formatter = new Intl.DateTimeFormat("en-US", {  
  year: "numeric",
```

```
    month: "long",
    day: "numeric"
  });

  console.log(formatter.format(new Date()));
```

Conceptual output:

September 3, 2026

Bangladesh Locale

```
const formatter = new Intl.DateTimeFormat("bn-BD", {
  year: "numeric",
  month: "long",
  day: "numeric"
});

console.log(formatter.format(new Date()));
```

Date + Time

```
const formatter = new Intl.DateTimeFormat("en-US", {
  dateStyle: "medium",
  timeStyle: "short"
});

console.log(formatter.format(new Date()));
```

Why Intl important?

Manual formatting:

```
const price = "$" + number.toFixed(2);
```

এটা শুধু একটি specific format।

International application হলে:

```
Intl.NumberFormat
```

অনেক বেশি reliable ।

7. `findLast()` & `findLastIndex()`

আগে:

```
array.find()
```

সামনের দিক থেকে প্রথম matching element খুঁজে ।

```
const numbers = [10, 20, 30, 40, 50];  
  
const result = numbers.find(number => number > 25);  
  
console.log(result);
```

Output:

```
30
```

কিন্তু যদি শেষ দিক থেকে খুঁজতে চাই?

Modern JavaScript:

```
numbers.findLast(number => number > 25);
```

Output:

```
50
```

findLast()

Syntax:

```
array.findLast(callback);
```

Example:

```
const users = [
  { name: "A", active: false },
  { name: "B", active: true },
  { name: "C", active: false },
  { name: "D", active: true }
];

const lastActiveUser = users.findLast(
  user => user.active
);

console.log(lastActiveUser);
```

Output:

```
{
  name: "D",
  active: true
}
```

`findLast()` শেষ দিক থেকে search করে এবং matching **value** return করে। না পেলে `undefined` return করে।

findLastIndex()

এটা matching element-এর index return করে।

```
const numbers = [10, 20, 30, 40, 50];

const index = numbers.findLastIndex(
  number => number > 25
);
```

```
console.log(index);
```

Output:

```
4
```

কারণ:

```
10  20  30  40  50
0   1   2   3   4
           ↑
        last match
```

find() vs findLast()

```
const numbers = [10, 20, 30, 40, 50];

numbers.find(n => n > 25);
// 30

numbers.findLast(n => n > 25);
// 50
```

findIndex() vs findLastIndex()

```
numbers.findIndex(n => n > 25);
// 2

numbers.findLastIndex(n => n > 25);
// 4
```

Search Direction

```
find()
→ → → → →

findLast()
← ← ← ← ←
```

Practical Example

ধরো order history:

```
const orders = [
  { id: 1, status: "pending" },
  { id: 2, status: "completed" },
  { id: 3, status: "cancelled" },
  { id: 4, status: "completed" }
];

const lastCompletedOrder = orders.findLast(
  order => order.status === "completed"
);

console.log(lastCompletedOrder);
```

Result:

```
{
  id: 4,
  status: "completed"
}
```

8. String.prototype.replaceAll()

Problem

আপে string-এর সব occurrence replace করতে অনেক সময় regex ব্যবহার করা হতো।

```
const text = "apple apple apple";  
  
const result = text.replace(/apple/g, "orange");  
  
console.log(result);
```

Modern JavaScript:

```
const result = text.replaceAll("apple", "orange");  
  
console.log(result);
```

Output:

```
orange orange orange
```

replace() vs replaceAll()

```
const text = "hello hello hello";
```

replace()

```
text.replace("hello", "hi");
```

Result:

```
hi hello hello
```

শুধু first match replace করে।

replaceAll()

```
text.replaceAll("hello", "hi");
```

Result:

```
hi hi hi
```

সব occurrence replace করে।

Case Sensitivity

```
const text = "Hello hello HELLO";  
  
console.log(text.replaceAll("hello", "Hi"));
```

সব `hello` case-insensitiveভাবে replace করবে না।

Matching সাধারণত exact/case-sensitive:

```
Hello → unchanged  
hello → replaced  
HELLO → unchanged
```

Case-insensitive replacement দরকার হলে regex ব্যবহার করতে পারো:

```
text.replaceAll(/hello/gi, "Hi");
```

`replaceAll()` with Function

`replaceAll()` শুধু string replacement-এর জন্য নয়।

```
const text = "10 20 30";  
  
const result = text.replaceAll(/\d+/g, match => {  
  return Number(match) * 2;  
});  
  
console.log(result);
```

Result:

20 40 60

`replaceAll()` returns a new string

Strings immutable।

```
const text = "hello hello";  
  
text.replaceAll("hello", "hi");  
  
console.log(text);
```

Original:

hello hello

কারণ string mutate হয়নি।

তাই:

```
const updated = text.replaceAll("hello", "hi");
```

Quick Comparison

Feature	Main Purpose
<code>Object.groupBy()</code>	Data grouping
<code>array.at()</code>	Index access, especially negative
<code>Object.hasOwn()</code>	Own property check
<code>Error.cause</code>	Preserve original error
<code>structuredClone()</code>	Deep clone
<code>Intl.NumberFormat</code>	Locale-aware number formatting
<code>Intl.DateTimeFormat</code>	Locale-aware date/time formatting

Feature	Main Purpose
<code>findLast()</code>	Last matching value
<code>findLastIndex()</code>	Last matching index
<code>replaceAll()</code>	Replace every occurrence

Feature-by-Feature Mental Model

`Object.groupBy()`

```
Array
↓
group condition
↓
Object
```

Example:

```
users
↓
role
↓
{
  admin: [...],
  user: [...]
}
```

`at()`

```
Array
↓
at(-1)
↓
last element
```

Object.hasOwn()

```
object  
↓  
own property?  
↓  
true / false
```

Error cause

```
High-level Error  
↓  
cause  
↓  
Original Error
```

structuredClone()

```
Original object  
↓  
deep clone  
↓  
Independent object
```

Intl

```
Raw value  
↓  
Locale + options  
↓  
Human-friendly format
```

`findLast()`

Array
← ← ← ←
search from end

`replaceAll()`

"foo foo foo"
↓
`replaceAll()`
↓
"bar bar bar"

Common Mistakes

Mistake 1 — `arr[-1]`

Wrong:

```
arr[-1];
```

Correct:

```
arr.at(-1);
```

Mistake 2 — Confusing `in` with `Object.hasOwn()`

```
"toString" in {};
```

can be:

```
true
```

কিন্তু:

```
Object.hasOwn({}, "toString");
```

is:

```
false
```

Mistake 3 — Thinking `structuredClone()` clones functions

```
structuredClone({  
  fn() {}  
});
```

এটা cloneable data হিসেবে function preserve করবে না।

Mistake 4 — Thinking spread is deep clone

Wrong assumption:

```
const copy = { ...original };
```

এটা nested object-এর full deep clone নয়।

Deep data clone দরকার হলে:

```
const copy = structuredClone(original);
```

Mistake 5 — `findLast()` returns index

Wrong:

```
array.findLast(...)
```

এটা **value** return করে।

Index দরকার হলে:

```
array.findLastIndex(...)
```

Mistake 6 — `replace()` automatically replaces everything

```
"foo foo".replace("foo", "bar");
```

শুধু first occurrence replace করে।

সবগুলো:

```
"foo foo".replaceAll("foo", "bar");
```

Mistake 7 — `Object.groupBy()` এবং `reduce()` একই জিনিস মনে করা

`groupBy()` specifically grouping-এর জন্য।

`reduce()` অনেক ধরনের accumulation করতে পারে।

Combined Real-World Example

ধরো API থেকে orders এসেছে:

```
const orders = [
  {
    id: 1,
    customer: "Rahim",
    category: "electronics",
    status: "completed",
    price: 50000
  }
]
```

```
    },  
    {  
      id: 2,  
      customer: "Karim",  
      category: "clothing",  
      status: "pending",  
      price: 3000  
    },  
    {  
      id: 3,  
      customer: "Hasan",  
      category: "electronics",  
      status: "completed",  
      price: 70000  
    }  
  ]  
};
```

Group by category

```
const grouped = Object.groupBy(  
  orders,  
  order => order.category  
);
```

Last order

```
const lastOrder = orders.at(-1);
```

Check property

```
Object.hasOwn(lastOrder, "price");
```

Find last completed order

```
const lastCompleted = orders.findLast(  
  order => order.status === "completed"  
);
```

Find last completed index

```
const lastCompletedIndex = orders.findLastIndex(  
  order => order.status === "completed"  
);
```

Clone order data

```
const clonedOrders = structuredClone(orders);
```

Format price

```
const formatter = new Intl.NumberFormat("en-BD", {  
  style: "currency",  
  currency: "BDT"  
});  
  
console.log(formatter.format(lastOrder.price));
```

এখানে একই application-এ modern JavaScript-এর কয়েকটি feature একসাথে কাজ করছে।

Practice Problems

Level 1 — Easy

Problem 1

```
const numbers = [10, 20, 30, 40, 50];
```

`at()` ব্যবহার করে বের করো:

- first element
 - last element
 - second last element
-

Problem 2

```
const user = {  
  name: "Sharlar",  
  age: 31  
};
```

`Object.hasOwn()` ব্যবহার করে check করে:

```
name  
email  
toString
```

Problem 3

```
const text = "JS is great. JS is powerful. JS is popular.";
```

`replaceAll()` ব্যবহার করে সব `JS`-কে `JavaScript` বানাও।

Level 2 — Medium

Problem 4 — Group Users

```
const users = [  
  { name: "A", role: "admin" },  
  { name: "B", role: "user" },  
  { name: "C", role: "admin" },  
  { name: "D", role: "user" }  
];
```

`Object.groupBy()` দিয়ে role অনুযায়ী group করো।

Expected structure:

```
{  
  admin: [...],
```

```
    user: [...]  
  }
```

Problem 5 — Last Active User

```
const users = [  
  { name: "A", active: false },  
  { name: "B", active: true },  
  { name: "C", active: false },  
  { name: "D", active: true }  
];
```

`findLast()` ব্যবহার করে last active user বের করো।

Problem 6 — Last Active Index

উপরের array থেকে `findLastIndex()` ব্যবহার করে last active user's index বের করো।

Level 3 — Hard

Problem 7 — Deep Clone

```
const state = {  
  user: {  
    name: "Sharar",  
    address: {  
      city: "Rangpur"  
    }  
  },  
  settings: {  
    theme: "dark"  
  }  
};
```

`structuredClone()` ব্যবহার করে independent copy তৈরি করো।

তারপর copy-এর:

```
user.address.city
```

change করো।

নিশ্চিত করো original object change হয়নি।

Problem 8 — Error Cause

একটি function তৈরি করো:

```
function loadUser() {  
  // simulate original error  
}
```

Original error catch করে নতুন error throw করো:

```
Failed to load user
```

এবং original error-কে:

```
cause
```

হিসেবে attach করো।

Problem 9 — International Price

```
const price = 1250000;
```

`Intl.NumberFormat()` ব্যবহার করে BDT currency format তৈরি করো।

Problem 10 — Combined Challenge

এই data:

```
const products = [
  {
    name: "Laptop",
    category: "electronics",
    price: 80000,
    active: true
  },
  {
    name: "Shirt",
    category: "clothing",
    price: 2500,
    active: false
  },
  {
    name: "Phone",
    category: "electronics",
    price: 50000,
    active: true
  },
  {
    name: "Pants",
    category: "clothing",
    price: 3000,
    active: true
  }
];
```

একটি program লিখো যেটা:

1. Category অনুযায়ী group করবে
2. Last product বের করবে
3. Last active product বের করবে
4. Last active product-এর index বের করবে
5. Products-এর deep clone তৈরি করবে
6. Price BDT format করবে
7. Product name-এ "Phone" থাকলে `replaceAll()` দিয়ে অন্য text বসাবে

Interview Questions

Beginner

1. `Object.groupBy()` কী?
2. `Object.groupBy()` এবং `reduce()` -এর difference কী?
3. `array.at(-1)` কেন useful?

4. `arr[-1]` কেন last element দেয় না?
5. `Object.hasOwn()` কী?
6. `Object.hasOwn()` এবং `in` operator-এর difference কী?
7. `replace()` এবং `replaceAll()` -এর difference কী?

Intermediate

1. `structuredClone()` কী?
2. Spread operator এবং `structuredClone()` -এর difference কী?
3. JSON deep cloning কেন problematic?
4. `find()` এবং `findLast()` -এর difference কী?
5. `findIndex()` এবং `findLastIndex()` -এর difference কী?
6. `Error.cause` কেন ব্যবহার করা হয়?
7. Error chaining কী?

Advanced

1. `Object.groupBy()` কী ধরনের object return করে?
2. `structuredClone()` কি functions clone করতে পারে?
3. `structuredClone()` কি circular references handle করতে পারে?
4. `Object.hasOwn()` কেন `obj.hasOwnProperty()` -এর চেয়ে safer?
5. `Intl.NumberFormat()` কেন manual number formatting-এর চেয়ে better?
6. `Intl.DateTimeFormat()` কীভাবে locale-aware formatting করে?

Quick Revision

```
// Group
Object.groupBy(items, item => item.category);

// Last element
items.at(-1);

// Second last
items.at(-2);

// Own property
Object.hasOwn(obj, "name");

// Error cause
throw new Error("High-level error", {
  cause: error
});

// Deep clone
const copy = structuredClone(original);
```

```

// Number formatting
new Intl.NumberFormat("en-US");

// Currency
new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD"
});

// Date formatting
new Intl.DateTimeFormat("en-US");

// Last matching value
items.findLast(item => condition);

// Last matching index
items.findLastIndex(item => condition);

// Replace everything
text.replaceAll("old", "new");

```

Final Mental Model

Modern JavaScript-এর এই features-গুলোকে এভাবে মনে রাখো:

```

Object.groupBy()
  ↓
Grouping data

at()
  ↓
Negative indexing

Object.hasOwn()
  ↓
Safe own-property checking

Error cause
  ↓
Error chaining / debugging context

```

`structuredClone()`



Deep data cloning

`Intl`



International formatting

`findLast()`



Last matching value

`findLastIndex()`



Last matching index

`replaceAll()`



Replace every occurrence

✓ Mastery Checklist

- [] `Object.groupBy()` বুঝি
- [] `Object.groupBy()` vs `reduce()` explain করতে পারি
- [] `at()` দিয়ে negative indexing করতে পারি
- [] `at(-1)` দিয়ে last element বের করতে পারি
- [] `Object.hasOwnProperty()` এবং `in` আলাদা করতে পারি
- [] `hasOwnProperty()` -এর limitation জানি
- [] `Error.cause` দিয়ে original error preserve করতে পারি
- [] `structuredClone()` দিয়ে deep clone করতে পারি
- [] shallow vs deep copy explain করতে পারি
- [] circular reference cloning বুঝি
- [] `structuredClone()` limitations জানি
- [] `Intl.NumberFormat()` ব্যবহার করতে পারি
- [] currency/percentage formatting করতে পারি
- [] `Intl.DateTimeFormat()` ব্যবহার করতে পারি
- [] `find()` vs `findLast()` বুঝি
- [] `findIndex()` vs `findLastIndex()` বুঝি

- `[] replace()` vs `replaceAll()` বুঝি
 - `[]` সবগুলো real-world code-এ ব্যবহার করতে পারি
-



Most Important Takeaways

সবগুলো feature-এর মধ্যে practical development-এর জন্য বিশেষভাবে এগুলো মাথায় রাখো:

```
arr.at(-1)
```

→ clean last-element access

```
Object.hasOwn(obj, key)
```

→ safe own-property check

```
structuredClone(data)
```

→ modern deep cloning

```
Object.groupBy(data, callback)
```

→ clean data grouping

```
arr.findLast(callback)
```

→ last matching item

```
arr.findLastIndex(callback)
```

→ last matching index

```
text.replaceAll(old, new)
```

→ replace all occurrences

```
new Error("message", {  
  cause: error  
})
```

→ preserve original error context

```
new Intl.NumberFormat(...)
```

→ locale-aware number/currency formatting

```
new Intl.DateTimeFormat(...)
```

→ locale-aware date/time formatting